

Chapter 3: People You May Know

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. People You May Know (PYMK) is **link prediction**: given the social graph up to now, which non-connected pairs are likely to connect *and* both value the connection? Generate a bounded pool of candidates, mostly friends-of-friends. Score them with a simple supervised model on graph, profile and activity features. Precompute and cache the lists for active users, refreshing lazily. Promote a **temporal GNN** only if it measurably beats the static model with recency features.

People You May Know design map

Move from product objective and graph scale to candidate generation, temporal scoring, serving, and feedback.



Minimum-scope principle

Begin with graph-based candidate generation and a pointwise or learning-to-rank baseline. Add temporal node memory only when recency features cannot capture the required lift.

Figure 1: People You May Know design map

1. Scope and scale

- **Input:** a user u opening a PYMK surface.
- **Model output:** $\hat{P}$ (a connection between u and v forms and is accepted within Δt) for candidate users v .
- **Service action:** a short ranked list (e.g. 10-20 people), excluding existing connections, blocked users and people who opted out.

Working assumptions: ~1B total users, ~300M daily active users, ~1,000 connections per user on average. The arithmetic drives the whole design:

Quantity	Estimate	Consequence
2-hop (friend-of-friend) paths per user	$1,000 \times 1,000 = 10^6$ (fewer unique people due to overlap; far more for high-degree users)	too many to score with a rich model

Quantity	Estimate	Consequence
scoring all 2-hop pairs for every DAU daily	$3 \times 10^8 \times 10^6 = 3 \times 10^{14}$	impossible, so the pool must be capped
cap at 2,000 candidates per user (e.g. by mutual-friend count)	6×10^{11} scores/day $\approx 7\text{M/s}$	feasible only for a cheap scorer
rich model on the top 200 3-hop	6×10^{10} /day $\approx 0.7\text{M/s}$ $\sim 10^9$	feasible, in batch only via sampling (random walks), never exhaustive

So: **exclude 1-hop, cap 2-hop, deduplicate, narrow cheaply, and score richly only at the end.**

2. Metrics

Metric	Measures	Pitfall
requests sent / received	activity	rewards spammy, low-quality outreach
accepted requests	mutual relevance	may still miss long-term value
completed connections that go on to interact	real network growth	sparse and delayed
Precision@K, mAP (offline)	quality of the visible list	offline proxy only
PR-AUC (offline)	separating rare positive edges	ignores list position

Starting choice: gate models offline with Precision@K / mAP and PR-AUC on future edges. Online, optimize **accepted connections per user**, with guardrails on request-rejection rate, “I don’t know this person” feedback, blocks and spam reports.

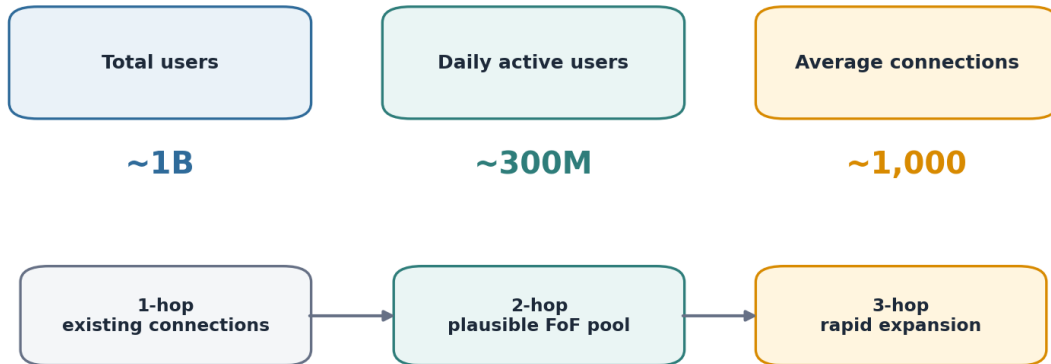
3. Model alternatives

Approach	Input	Strength	Cost
pointwise classifier / learning-to-rank (GBDT or MLP)	user, candidate and pair features, including hand-made graph features	easy to train, debug and serve	graph structure only through features you engineer
static graph model (node2vec / GraphSAGE embeddings) + recency features	graph snapshot + time-since features	learns structure automatically	a snapshot loses event order
temporal GNN (e.g. TGN, Rossi et al., 2020)	the stream of timestamped graph events	models <i>how relationships evolve</i>	per-node state, complex training and serving

Promotion rule: keep the simpler model unless event order and memory produce a measurable lift (e.g. $\geq 2\%$ relative mAP offline and a significant gain in accepted connections online) that pays for the extra state and complexity.

Scale changes the architecture

Illustrative capacity-planning assumptions, not figures from any specific platform.

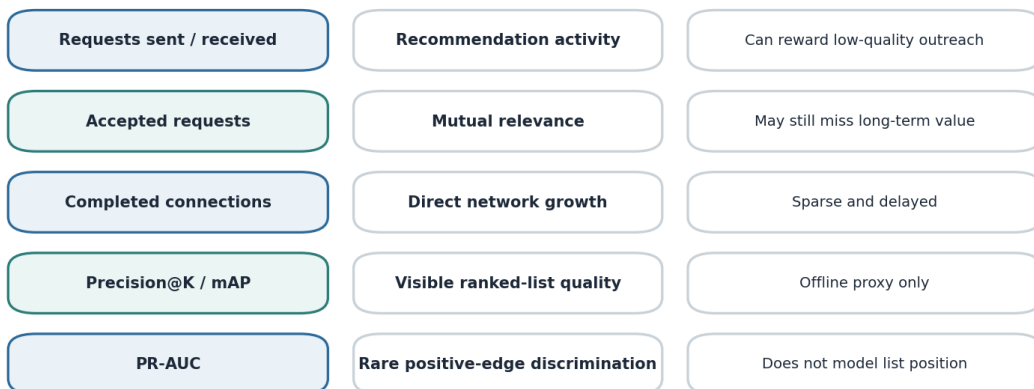


Design consequence: exclude current connections, cap graph expansion, deduplicate candidates, and rank before expensive pair modeling.

Figure 2: Scale constraints

Metrics separate attraction from mutual value

A request sent is not the same outcome as an accepted or durable connection.

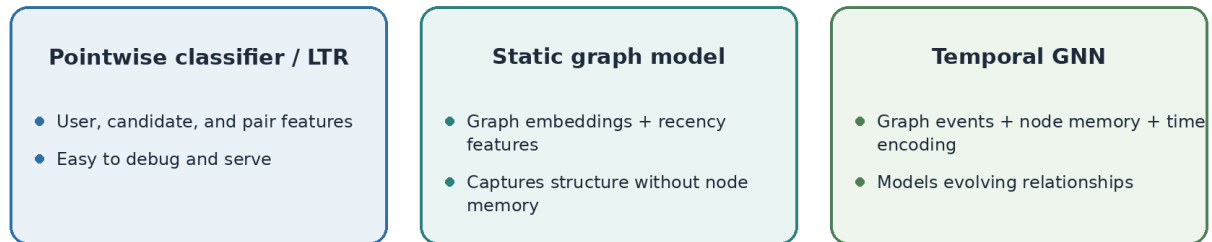


Starting recommendation: gate the model with top-K and rare-positive metrics offline, then optimize accepted or completed connections online with safety and spam guardrails.

Figure 3: Metrics

Baseline versus graph-aware models

The temporal GNN is the richer proposal, not the required starting point.



Promotion rule

Keep the simpler model unless event order and memory produce measurable lift that justifies state, training, and serving complexity.

Figure 4: Model alternatives

Build time-aware social affinity from several signal families

Mutual-connection count alone may miss recency, engagement, and repeated non-response.

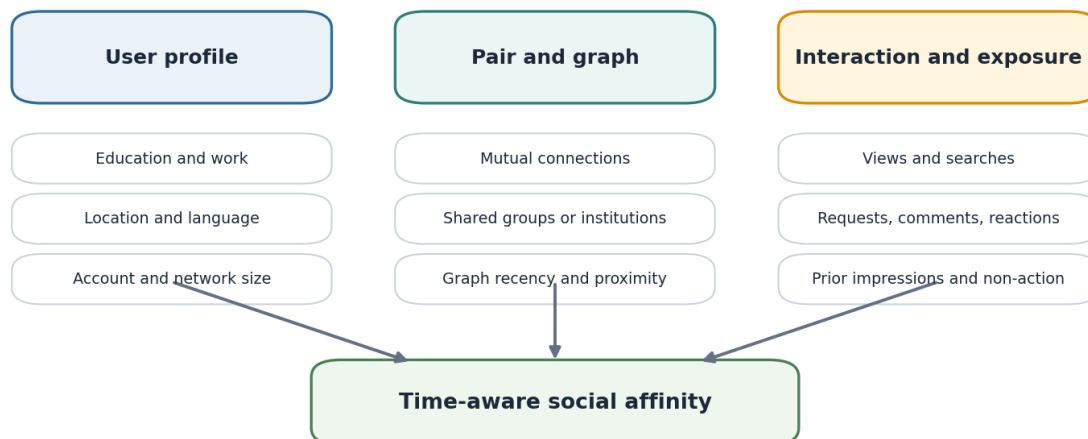


Figure 5: Features and affinity

4. Features: time-aware social affinity

Mutual-friend count alone is a strong but blunt signal. It misses recency and engagement, and it ignores people who have been shown repeatedly and ignored.

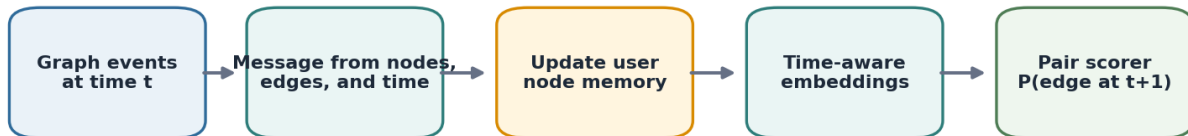
Family	Examples
user profile	school, employer, location, language, account age, network size
pair and graph	mutual connections (count and Adamic-Adar $\sum_{w \in N(u) \cap N(v)} 1/\log N(w) $, which down-weights mutuals who know everyone), shared groups or institutions, graph distance, recency of the mutual connections
interaction and exposure	profile views and searches between u and v , comments or reactions on each other's posts, times v was already shown to u without action

Leakage trap. Every graph feature must be computed on the graph **before** the prediction time. Computing mutual-friend counts on today's graph for a training pair that connected last week lets the model see the edge it's supposed to predict, and its consequences.

5. The temporal GNN

Temporal GNN edge-prediction flow

Events update user memory, which produces time-aware embeddings for future-edge scoring.



Possible memory contents



The memory updater can be a GRU or LSTM cell (as in TGN); the exact recurrent mechanism is an implementation choice.

Figure 6: Temporal GNN

In a TGN-style model, each graph event (a new connection, message or profile view at time t) creates a **message** from the two endpoints' states and the time gap. The message updates each node's **memory** vector through a GRU (or LSTM) cell. A temporal attention layer

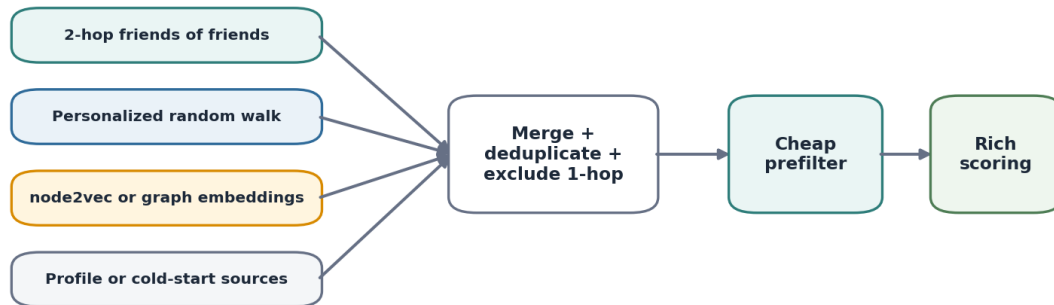
over recent neighbors then produces a **time-aware embedding**, and a pair scorer predicts $P(\text{edge}(u, v) \text{ at } t + 1)$.

The memory can hold recent graph changes, direct views and searches, engagement across the shared network, and time since meaningful events. The cost: every node carries state that must be updated in stream order, stored, and kept consistent between training and serving.

6. Candidate generation

Candidate generation should combine graph sources

Use inexpensive graph methods to produce a bounded pool before expensive scoring.



Friends-of-friends is usually the highest-yield source; the others add coverage and cold-start candidates.

Figure 7: Candidate generation

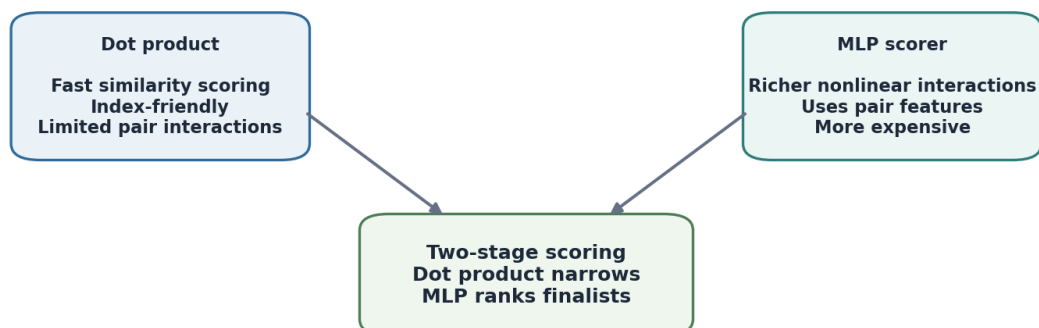
Source	Finds	Note
2-hop friends-of-friends	people with mutual connections	usually the highest-yield source; cap per user by mutual count
personalized random walk (PageRank with restart)	well-connected people 2-3 hops away	samples 3-hop without enumerating it
node2vec / graph-embedding ANN	structurally similar people, even without mutuals	catches communities
profile / cold-start sources	same school, company or city; contact-book matches (with consent)	for new users with few edges

Merge, deduplicate, **exclude existing connections, pending requests, blocked users and opted-out users**, then narrow with a cheap score.

7. Pair scoring

Dot product and MLP solve different scoring constraints

Use one or both depending on retrieval scale and pair-feature requirements.



Starting recommendation: use fast graph or embedding scores for candidate reduction, then use an MLP only where richer pair features provide measurable ranking lift.

Figure 8: Pair scoring

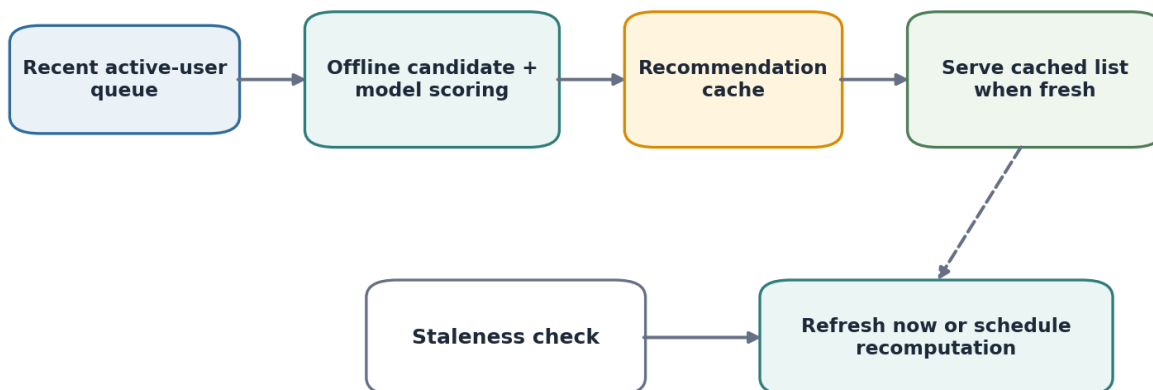
Scorer	Pros	Cons
dot product of embeddings	index-friendly, very cheap	no pair features, limited interactions
MLP on [user, candidate, pair features]	uses mutual counts, interaction history, exposure	more expensive per pair

Two stages: the dot product (or mutual count) narrows ~2,000 candidates to ~200, and the MLP ranks those finalists. Use the MLP only if pair features add measurable ranking lift.

8. Serving: precompute, cache, refresh lazily

Precompute, cache, and refresh lazily

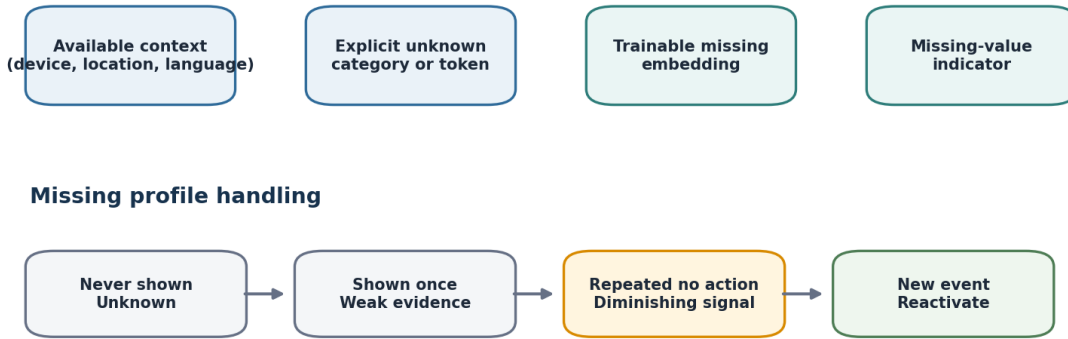
Avoid recomputing recommendations after every small graph event.



A lightweight ranker may prefilter before expensive scoring, rerank cached candidates with fresh features, or perform both roles.

Missing profiles and repeated exposure need explicit treatment

Missingness is information; non-action becomes stronger only after repeated opportunities.



New graph changes, profile views, searches, or shared-network engagement can restore a previously suppressed candidate.

Figure 10: Missing data and exposure

Non-action is a graded signal:

never shown → unknown
 shown once → weak negative evidence
 shown repeatedly → diminishing score (e.g. multiply by $\gamma^{(\text{times shown})}$, $\gamma \approx 0.7$), then temporary
 new event → reactivate (new mutual friend, profile view, search, shared-group activity)

Delayed feedback: a request can be accepted days later. Evaluate and label with a window (e.g. accepted within 14 days), and don't train on pairs whose window hasn't closed.

10. Negatives and online evaluation

Negative type	Definition	Bias
random non-edges	any unconnected pair	too easy: most random pairs are obviously unrelated
graph-hard	friends-of-friends who didn't connect	realistic, but some are future positives
exposure	shown in PYMK, no action	the most relevant; depends on the old model's choices
time-aware	no edge within the label window	the correct labeling window, but delayed

Mix them: exposure + graph-hard for relevance, plus some random for calibration.

Online: *interleaving* compares two rankers quickly by merging their lists and counting accepts from each. *A/B tests* measure product impact.

Note. > **Beyond the book: network interference in A/B tests.** A PYMK change in treatment also affects control users, who receive the treatment users' requests. Randomize by graph clusters (ego-network or community randomization) or measure both sent and received effects. Otherwise the estimated lift is biased.

Training negatives and online comparison

Which non-connected pairs count as negatives shapes what the model learns.

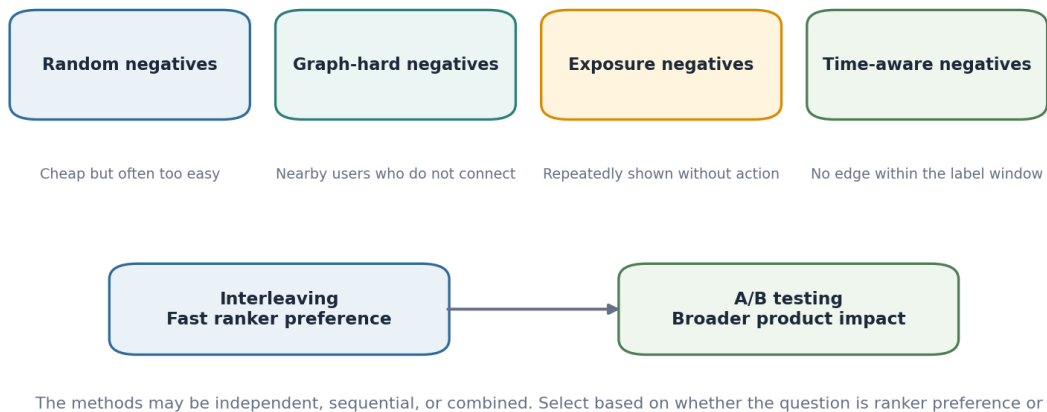


Figure 11: Negatives and evaluation

Note. > **Beyond the book: privacy.** Candidate sources like contact uploads or co-location can reveal sensitive relationships (a therapist, an ex). Respect privacy settings, avoid explaining recommendations with sensitive signals, and give users an easy “don’t suggest” control.

11. Reference architecture

Offline: graph and interaction event logs → FoF / random-walk / embedding sources → merge + dedupe + exclusions → pointwise (later static or temporal) scoring → cache writer with staleness metadata. **Online:** request → read cached candidates → fresh-feature light re-rank → exposure and policy filter → top-K.

12. Decisions and when I’d change them

Area	Starting choice	Switch when (measured)
product metric	accepted connections per user	long-term interaction data shows accepted-but-dormant links dominate → optimize connections that interact
candidates	capped 2-hop + random walk + cold-start sources	recall@K of the pool drops below target for low-degree users → add embedding ANN
model	pointwise GBDT/MLP with graph features	a temporal GNN beats it by $\geq \sim 2\%$ relative mAP offline and wins online
scoring	cheap narrowing → MLP finalists	the MLP adds no lift over the cheap score → drop it

Area	Starting choice	Switch when (measured)
serving	cached, lazy refresh, light re-rank	measured staleness hurts accepts (e.g. many suggestions of already-connected people) → refresh more often
ignored suggestions	decay by exposure count + reactivation	analysis shows re-shown candidates convert as often as fresh ones → weaker decay
evaluation	Precision@K / mAP / PR-AUC → interleaving → cluster-randomized A/B	-

13. Questions and answers

Q: Why not just rank by mutual-friend count?

It's a strong baseline, and should be the control arm. But it ignores recency (mutuals from 10 years ago), hubs (a mutual who knows everyone), engagement signals and prior ignores. Adamic-Adar and a learned model typically beat it.

Q: Should 3-hop candidates ever be used?

For users with few connections, yes, via random walks that sample 3-hop neighbors in proportion to connectivity. Enumerating 3-hop is never feasible.

Q: How much node memory can a temporal GNN afford?

At 10^9 nodes, 100 float32 values per node is already 400 GB of state. Memory has to be small (e.g. 32-64 dims), compressed, or kept only for active users.

Q: How should missing profile values be handled?

With an explicit unknown token and a missing indicator per field, so the model learns what missingness means (often "new user" or "privacy-conscious user"), rather than imputing a default value.

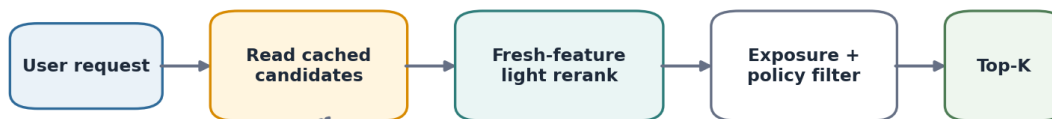
Q: Interleaving or A/B test?

Interleaving to pick between rankers quickly (it needs far less traffic). A cluster-randomized A/B test to measure the effect on network growth before launch.

Reference architecture

One production-balanced design under the current assumptions.

ONLINE SERVING



OFFLINE GRAPH LEARNING AND REFRESH



Starting recommendation: graph-based bounded candidate generation, a simple supervised scorer first, cached recommendations with lazy refresh, and temporal GNN promotion only after measurable incremental value.

Figure 12: Reference architecture